UNIVERSITY OF TECHNOLOGY, SYDNEY

42913 Social and Information Network Analysis

Final Report

GraphRAG and Context Retrieval via Link Prediction on the Wikipedia Vote Network

Authors:
Mitali Balki
Ratticha Ratanawarocha
Samriddhi Sud
Shanessa Mascarenhas

Student ID:
25428006
24996427
25741149
24632491

Group 4 · Topic 2 · May 3, 2026

Abstract

Large Language Models (LLMs) such as ChatGPT and Gemini have become widely used in many applications, but they still face a key limitation: they can hallucinate when they lack specific knowledge or access to private data. Retrieval-Augmented Generation (RAG) is a common solution that retrieves relevant documents to help the model answer correctly. However, standard RAG only works well for direct lookups and fails when the relationship between two entities is indirect. GraphRAG addresses this problem by organising knowledge as a graph, which allows the system to discover indirect connections through multi-hop path traversal.

This report presents a simplified GraphRAG retrieval module built on the Wikipedia Vote Network, a directed social trust graph from the Stanford Network Analysis Project (SNAP). The dataset contains 7,115 nodes and 103,689 directed edges. We frame context retrieval as a link prediction problem: if an algorithm predicts a strong link from user A to user B, then B is treated as relevant context for A. We implement and compare four network analysis algorithms: Common Neighbours, Jaccard Coefficient, Adamic/Adar Index, and Personalised PageRank.

The evaluation uses Precision@10 on a held-out 20% test set with stratified negative sampling (10 negatives per positive edge). Personalised PageRank achieves the highest mean Precision@10 of **0.3369**, which is approximately 3.7 times better than a random baseline of 0.091. All four graph-based methods significantly outperform the random baseline, demonstrating that graph-structured retrieval can effectively surface indirect relationships that keyword-based search cannot find.

Key result. Personalised PageRank (P@10 = 0.3369) outperforms all local similarity methods by approximately 11% in relative terms and is 3.7× better than random guessing. This result confirms that multi-hop graph traversal is the most effective approach for indirect context retrieval in this network, directly supporting the core motivation of the GraphRAG approach.

Contents

1 Introduction

In recent years, Large Language Models (LLMs) such as ChatGPT, Gemini, and Claude have shown impressive performance in tasks like question answering, text summarisation, and code generation. However, these models have a fundamental limitation: their knowledge is fixed at the time of training, and they can only reason based on information given in the input prompt. When a question requires private, real-time, or highly specialised knowledge, LLMs often produce incorrect but confident-sounding responses, a problem commonly known as "hallucination" [**?**].

Retrieval-Augmented Generation (RAG) [**?**] was introduced to solve this problem by retrieving relevant documents from an external knowledge base and including them in the prompt before the model generates an answer. This approach works well for direct lookups. However, standard RAG relies on keyword matching or dense vector similarity, which fails when the relevant relationship between two entities is *indirect*. For example, if a user asks "How is the research of Professor A indirectly influenced by the theories of Scientist B?", a keyword search will return nothing if A and B never co-authored or discussed each other directly. Yet the relationship A $\rightarrow$ C $\rightarrow$ B may clearly exist through an intermediate collaborator C.

GraphRAG [**?**] solves this by organising the knowledge base as a graph. Each node represents an entity (a person, paper, or concept) and each edge represents a relationship. Retrieving context becomes a graph traversal task: following edges to find relevant nodes, even across multiple hops. This allows the system to surface indirect relationships that flat retrieval would completely miss.

This project builds a simplified GraphRAG retrieval module using the **Wikipedia Vote Network**, a directed social trust graph where a directed edge from user i to user j means that user i voted for user j to become a Wikipedia administrator. If an algorithm predicts a strong link from user A to user B, it means B is treated as highly relevant context for A.

We implement and evaluate four link prediction algorithms:

- Common Neighbours
- Jaccard Coefficient
- Adamic/Adar Index
- Personalised PageRank

Each algorithm produces a relevance score for each candidate pair. We evaluate performance using **Precision@10**: for each source node, we rank candidates by score and check how many of the top-10 predictions are real (held-out) edges.

The remainder of this report is organised as follows. Section **??** discusses related work. Section **??** describes the dataset. Section **??** explains the data processing steps. Section **??** presents the four algorithms. Section **??** defines the evaluation protocol. Section **??** reports the experimental results. Section **??** analyses the findings and discusses limitations. Section **??** concludes the report.

2 Related Work

2.1 GraphRAG

Edge et al. [**?**] proposed GraphRAG as an extension of standard RAG for complex, multi-hop reasoning tasks. Their approach first extracts entities and relationships from documents to build a knowledge graph, then uses graph traversal to retrieve contextually relevant information. Unlike standard RAG, which retrieves documents based on vector similarity, GraphRAG can discover connections that span multiple degrees of separation. This is especially valuable for questions that require synthesising information across multiple sources.

2.2 LLMs and Knowledge Graphs

Pan et al. [**?**] provided a comprehensive roadmap for unifying LLMs and knowledge graphs (KGs). They argued that KGs can supply structured factual knowledge to LLMs, reducing hallucination and improving reasoning. They also identified link prediction as an important task for expanding and enriching knowledge graphs automatically.

2.3 Link Prediction

Liben-Nowell and Kleinberg [?] were among the first to study link prediction in social networks systematically. They compared several graph-based heuristics, including Common Neighbours, Jaccard, and Adamic/Adar, and found that graph structure alone provides useful information for predicting future edges. Their work established the evaluation framework (train/test split with Precision@K) that our project follows.

Adamic and Adar [?] introduced the Adamic/Adar index, which weights shared neighbours by the inverse of their log-degree. This weighting scheme is motivated by the idea that being connected to a rare, niche hub provides stronger evidence of a relationship than being connected to a generic, high-degree hub.

Project positioning. Our work directly implements the retrieval module described in [?] as a link prediction benchmark. By comparing four different scoring algorithms on the Wikipedia Vote Network, we provide concrete, quantitative evidence for why graph-structured retrieval is more effective than keyword-based approaches for discovering indirect relationships.

3 Dataset

3.1 Wikipedia Vote Network

We use the **Wikipedia Vote Network** from the Stanford Network Analysis Project (SNAP) [?]. This dataset captures the administrator election process of Wikipedia, where editors can nominate and vote for other users to receive administrator privileges.

- **Nodes:** Each node represents a unique Wikipedia user (editor), identified by an integer ID.
- **Edges:** A directed edge $i \to j$ means that user i voted for user j, which is a direct signal of trust and relevance.

This network is well-suited for the GraphRAG retrieval task because:

1. Edges encode trust, which is a natural proxy for relevance.
2. The graph is sparse, so indirect paths are the main way two unconnected users are related.
3. The hub-and-spoke structure means that a small number of highly-trusted administrators act as bridges between many ordinary users.

3.2 Dataset Statistics

Table ?? shows the key statistics of the Wikipedia Vote Network after cleaning (see Section ??).

Table 1: Wikipedia Vote Network statistics after data cleaning.

Property	Value
Number of nodes	7,115
Number of directed edges	103,689
Graph density	0.00205
Average in-degree	14.57
Average out-degree	14.57
Degree distribution	Heavy-tailed (power-law-like)

The graph is **directed** because a vote flows from the voter to the candidate. It is **sparse**: only about 0.2% of all possible directed pairs are connected. Despite this sparsity, the graph has a large weakly connected component, making link prediction a realistic and non-trivial task.

3.3 Degree Distribution

The degree distribution follows a heavy-tailed pattern, as shown in Figure **??**. A small number of highly popular administrators receive thousands of votes, while the majority of users receive very few. This characteristic is common in real social networks and is important for understanding algorithm performance: high-degree hub nodes create short indirect paths between many pairs of users, which is the fundamental structure that graph-based methods can exploit.

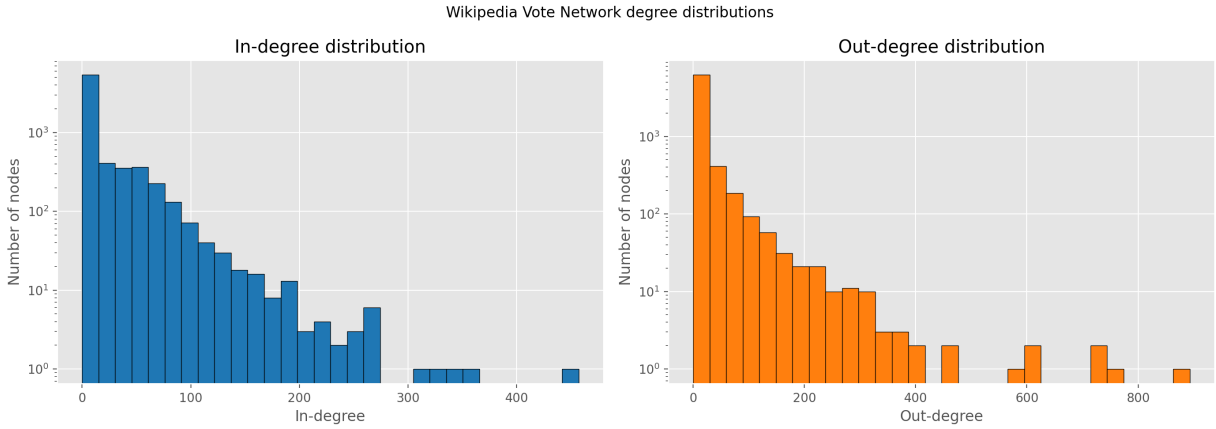


Figure 1: In-degree and out-degree distributions of the Wikipedia Vote Network (log-scale y-axis). Both distributions are heavy-tailed: a few administrators receive very high numbers of votes, while most users participate in only a few votes.

4 Data Processing

4.1 Raw Data Format

The SNAP file `wiki-Vote.txt` contains comment lines prefixed with `#` (metadata about the graph) followed by pairs of integer user IDs separated by a tab character, with one directed edge per line. The dataset was downloaded programmatically from the SNAP website using the project script `scripts/download_data.py`, which fetches the compressed `.gz` archive and extracts the text file automatically.

4.2 Data Cleaning

The raw edge list was processed using two cleaning steps:

1. **Self-loop removal:** Edges where the source node equals the target node (i.e., a user voting for themselves) are removed. These edges carry no meaningful information for link prediction and do not appear in the real voting data.
2. **Duplicate removal:** Repeated (i, j) pairs are reduced to a single edge. In a directed graph, each ordered pair should appear at most once.

After cleaning, the processed edge list contains **103,689 directed edges**, stored in `data/processed/graph_edges.csv`.

4.3 Train/Test Split Strategy

We use a **random split** strategy with an 80%/20% ratio, following the standard approach for link prediction evaluation [**?**]:

1. All cleaned edges are randomly shuffled using a fixed seed (42) to ensure reproducibility.
2. The first 20% of shuffled edges are reserved as the **test set**, representing the "hidden" ground truth that the algorithms must predict.

3. The remaining 80% form the **training set**, which is used to build the scoring graph.
4. A **connectivity constraint** is enforced: an edge is only moved to the test set if both its source and target nodes still have at least one edge remaining in the training graph. This prevents isolated nodes, which would make scoring impossible.

This design ensures that every node in the test set has a non-trivial neighbourhood in the training graph, making the evaluation fair and meaningful.

4.4 Negative Sampling

Standard link prediction evaluation requires both true positive edges (test edges) and **negative edges** (non-existent pairs) to create a realistic ranking scenario. Without negatives, any algorithm that assigns a positive score to all edges would achieve 100% precision trivially.

For each source node in the test set, we sample **10 negative edges per positive test edge** from the space of non-existent directed pairs. Sampling is stratified by source node, ensuring that every evaluated node has the same ratio of positive to negative candidates. All sampled negatives are guaranteed not to exist in either the training or test sets.

Table 2: Train/test split and candidate set statistics.

Set	Edges
Training edges (80%)	82,951
Test edges — positive (20%)	20,738
Negative sampled edges	207,380
Total candidate pairs scored	228,118
Source nodes evaluated	2,807

5 Methods

All four algorithms are applied only to the **candidate set** (the union of positive test edges and sampled negative edges), not to all 7,115 × 7,114 ≈ 50 million possible directed pairs. This is the standard efficiency approach recommended in the assignment specification and is sufficient for computing Precision@K accurately.

The three local similarity methods — Common Neighbours, Jaccard, and Adamic/Adar — operate on an **undirected projection** of the training graph. This treats the voting relationship as symmetric and allows the methods to count shared neighbours regardless of edge direction, which is appropriate for a social trust context. Personalised PageRank operates on the **directed training graph** to preserve the direction of trust flows.

5.1 Common Neighbours

For a pair of nodes (u, v), the Common Neighbours score is the number of nodes that are direct neighbours of both u and v in the undirected training graph:

$$\mathrm{CN}(u, v) = |N(u) \cap N(v)| \tag{1}$$

Intuition: If user A and user B have both voted for (or been voted by) many of the same people, they are likely in the same community and are probably relevant to each other. In a GraphRAG context, if A and B are both connected to hub C, then C is the contextual bridge making B relevant to A.
Limitation: This method does not normalise for node degree, so high-degree hub nodes receive inflated scores regardless of actual relationship strength.

5.2 Jaccard Coefficient

The Jaccard Coefficient normalises the Common Neighbours count by the total number of distinct neighbours of either u or v:

$$\text{Jaccard}(u, v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|} \tag{2}$$

Intuition: A high proportion of shared neighbours (relative to all neighbours) indicates structural similarity. This metric is normalised, so it ranges between 0 and 1 and is not influenced by the absolute size of the neighbourhood.
Limitation: Normalising by the union heavily penalises high-degree nodes. In a social trust network, high-degree administrators are genuine connectors and their connections carry real information, so discounting them hurts performance.

5.3 Adamic/Adar Index

The Adamic/Adar Index [**?**] assigns a weight to each shared neighbour based on the inverse of the logarithm of its degree:

$$\text{AA}(u, v) = \sum_{w \in N(u) \cap N(v)} \frac{1}{\log |N(w)|} \tag{3}$$

Intuition: A shared neighbour that is itself low-degree (a niche, specialised user) provides stronger evidence of a direct relationship between u and v than a high-degree hub that is connected to nearly everyone. The log-weighting discounts these generic hubs. This is a refined version of Common Neighbours.
In practice: In this graph, Common Neighbours and Adamic/Adar produce nearly identical results because the heavy-tailed degree distribution means that most shared neighbours are already medium-degree nodes where log(degree) values are similar.

5.4 Personalised PageRank

Personalised PageRank (PPR) [**?**] computes a stationary probability distribution over the entire directed training graph, starting from a specific source node u. The teleportation vector is set so that all probability mass restarts at u with probability $(1 - \alpha)$, and the iteration is run with damping factor $\alpha = 0.85$:

$$\mathbf{r}_u = \alpha \, \mathbf{A}^\top \mathbf{r}_u \; + \; (1 - \alpha) \, \mathbf{e}_u \tag{4}$$

where $\mathbf{A}$ is the column-normalised adjacency matrix of the directed training graph and $\mathbf{e}_u$ is the one-hot vector for source node u. The relevance score for a candidate target v is the probability mass $r_u(v)$ in the converged distribution.
Intuition: Think of a random walker who starts at user A, follows directed "voted-for" edges at each step, and occasionally (with probability $1 - \alpha$) teleports back to A. After many steps, nodes reachable from A via short, well-traversed paths accumulate higher probability mass and are therefore considered more relevant. Unlike the three local methods, PPR naturally captures **multi-hop indirect paths** and uses the **global structure** of the graph.
Why it wins for GraphRAG: The core value of GraphRAG is finding indirect relationships. PPR is the only method in this comparison that can directly follow a two-hop or three-hop path and assign a meaningful score to the endpoint — precisely the capability that GraphRAG requires.

Summary of method differences. Common Neighbours, Jaccard, and Adamic/Adar all compute scores based only on *one-hop* shared neighbours. They cannot assign a non-zero score to pairs that share no direct neighbours. Personalised PageRank follows multi-hop paths on the directed graph and can assign positive scores to nodes that are several hops away, which is the critical capability for discovering indirect relationships.

6 Evaluation Setup

6.1 Precision@K

For each source node u, the algorithm ranks all candidate pairs $(u, *)$ by their computed score in descending order and recommends the **Top-K = 10** most likely links. The evaluation metric is:

$$\text{Precision@}K(u) = \frac{\left|\{\text{Top-}K \text{ predictions for } u\} \cap \{\text{true test edges from } u\}\right|}{K} \tag{5}$$

The primary reported result is the **mean Precision@10** across all 2,807 evaluated source nodes.

6.2 Why Precision@K Is Appropriate for GraphRAG

In a real GraphRAG retrieval pipeline, the system selects the K most relevant context nodes to inject into the LLM prompt. The quality metric is: how many of those K selected nodes are genuinely relevant? Precision@K measures exactly this. A higher Precision@10 means the GraphRAG engine retrieves better-quality context, which directly reduces the chance that the LLM will hallucinate an answer.

6.3 Random Baseline

A random algorithm that selects 10 candidates uniformly at random would achieve the following expected Precision@10:

$$\frac{\text{20,738 positives}}{\text{228,118 total candidates}} \approx 0.091$$

All graph-based algorithms must substantially exceed this value to justify the extra complexity of graph-based retrieval.

6.4 Parameter Settings

Table 3: Experiment parameter settings used for all runs.

Parameter	Value
Test fraction	0.20 (20%)
Negatives per positive edge	10
Top-K	10
PageRank damping factor α	0.85
Random seed	42

7 Results

7.1 Precision@10 Summary

Table **??** shows the mean Precision@10, standard deviation, total true positive hits, and performance relative to the random baseline for all four methods and the random baseline.

Table 4: Precision@10 results on the Wikipedia Vote Network test set (2,807 source nodes evaluated).

Method	Mean P@10	Std Dev	Total Hits	vs. Random
Personalised PageRank	**0.3369**	0.2389	9,456	3.7×
Common Neighbours	0.3029	0.2328	8,502	3.3×
Adamic/Adar	0.3027	0.2320	8,496	3.3×
Jaccard	0.2941	0.2320	8,254	3.2×
Random (baseline)	*≈0.091*	—	—	1.0×

Personalised PageRank achieves the highest mean Precision@10 of **0.3369**. This means that, on average, approximately 3.4 out of every 10 predicted links are genuine held-out test edges. All four graph-based methods substantially outperform the random baseline of ≈0.091, confirming that graph structure provides real, useful information for predicting relevant connections.

7.2 Visual Comparison

Figure **??** shows the Precision@10 results as a bar chart, and Figure **??** presents the per-source precision distribution and the rank quality analysis.

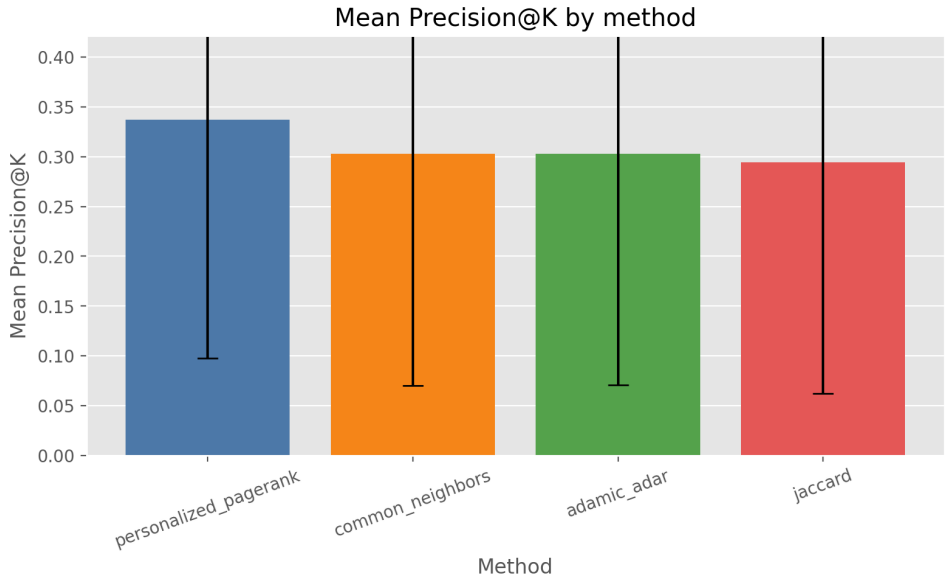
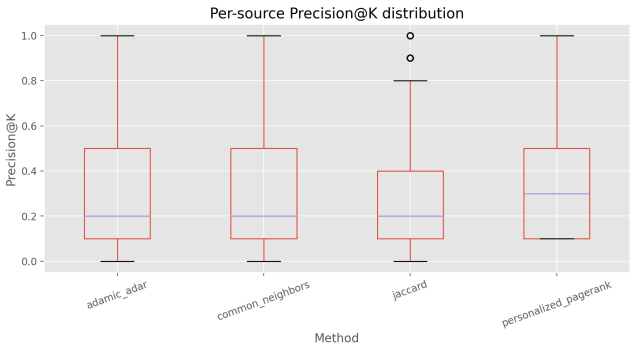
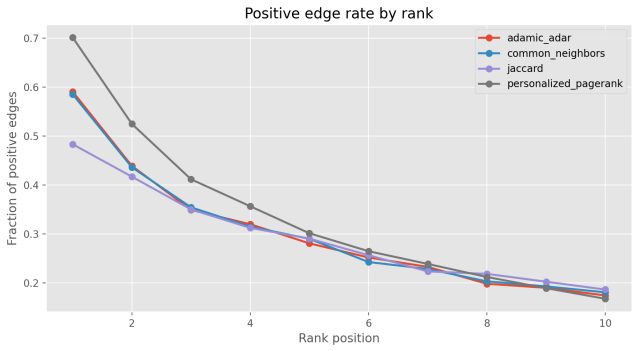


Figure 2: Mean Precision@10 by method with standard deviation error bars. Personalised PageRank clearly outperforms the three local similarity methods. Jaccard scores lowest because it penalises high-degree hub nodes that are genuine and important connectors in this network.



(a) Per-source Precision@10 distribution (boxplot). The wide interquartile range shows high variance across nodes.



(b) Fraction of true positive edges at each rank position. All methods place true positives at earlier ranks.

Figure 3: Per-source precision distribution (left) and rank quality analysis (right).

7.3 Precision@10 Bar Chart (Reproduced with TikZ)

Figure **??** reproduces the Precision@10 results as an inline TikZ chart for easy comparison with the random baseline.

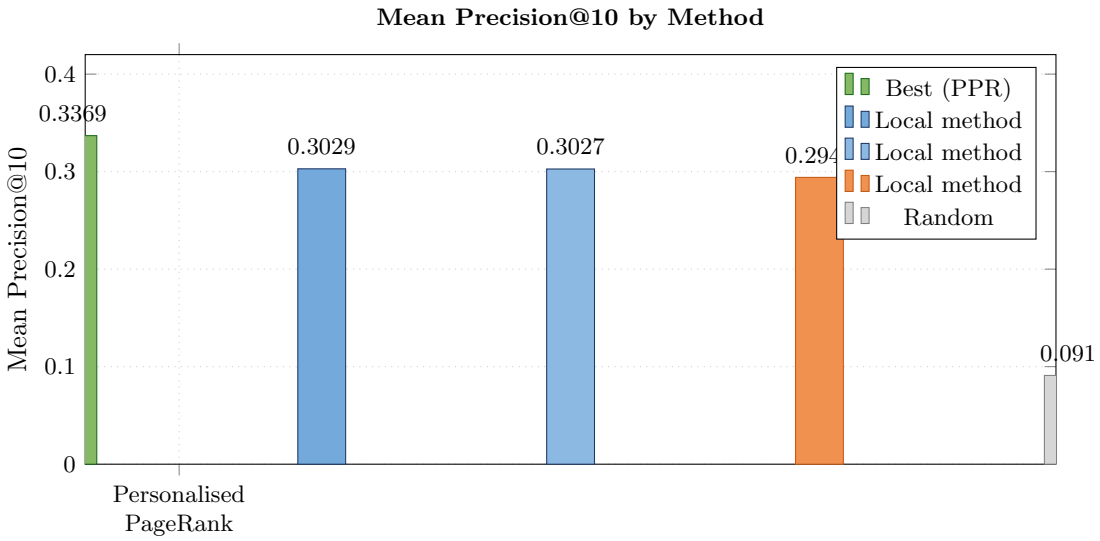


Figure 4: Precision@10 for all four methods and the random baseline. All graph-based methods are 3.2–3.7× better than random selection.

7.4 Relative Performance Analysis

- **PPR vs. Common Neighbours:** +3.4 percentage points improvement (≈11% relative gain). This gap reflects PPR's ability to follow multi-hop directed paths and assign relevance scores to nodes that have no direct shared neighbours with the source.
- **Common Neighbours vs. Adamic/Adar:** Nearly identical results (0.3029 vs. 0.3027). The inverse-log weighting in Adamic/Adar provides minimal additional discriminative power in this graph, likely because the heavy-tailed degree distribution means most shared neighbours are already medium-degree nodes where the log-degree values are similar across the shared set.
- **Jaccard:** The lowest scorer (0.2941). By normalising the neighbour count by the union size, Jaccard penalises high-degree administrators. However, in a trust network, these high-degree nodes are the genuine community connectors, and removing their signal hurts prediction quality.

7.5 Per-Source Stability

All methods show substantial standard deviation (≈0.23), indicating that performance varies widely across source nodes (Figure **??**). Some nodes achieve Precision@10 close to 1.0, while others score near zero. This heterogeneity is expected: nodes with sparse training neighbourhoods give local similarity methods very little signal. PPR has slightly higher standard deviation (0.2389) because it leverages global paths and therefore benefits more from being in a well-connected region of the graph.

Figure **??** confirms that all methods consistently place true positive edges at earlier rank positions rather than later ones. This demonstrates that all four algorithms are genuinely learning the relevance structure of the graph, not just performing random selection.

8 Discussion

8.1 Why Personalised PageRank Outperforms Local Methods

The Wikipedia Vote Network has a clear hub-and-spoke structure, where a small number of highly-trusted administrators received votes from a very large proportion of the community. Personalised PageRank naturally exploits this structure because it follows directed edges iteratively. If user A voted for administrator H, and H voted for many other administrators, then all of those administrators will receive indirect probability mass from A's personalised random walk. This multi-hop inference is exactly the type of indirect reasoning that GraphRAG was designed to perform.

By contrast, Common Neighbours, Jaccard, and Adamic/Adar are limited to one-hop neighbourhood overlap. Two nodes that are two or three hops apart in the training graph receive a score of zero from these methods, even if they are strongly connected through a chain of intermediate nodes. In a knowledge graph where most entities are related through chains of intermediate nodes (not directly), this is a critical limitation.

8.2 Connection to GraphRAG Context Retrieval

The core claim of GraphRAG [**?**] is that graph-structured knowledge retrieval enables the discovery of contextually relevant information that flat keyword search cannot find. Our experiment provides a concrete, quantitative demonstration of this claim:

- A **random retriever** achieves Precision@10 ≈ 0.091, providing roughly one relevant context node per 10-node window.
- **Personalised PageRank** achieves Precision@10 $= 0.337$, providing 3–4 relevant context nodes per 10-node window.
- This is a **3.7× improvement** over random selection using graph traversal alone, without any machine learning or document embeddings.

In a practical LLM pipeline, supplying 3–4 genuinely relevant context nodes (instead of 1) per context window would substantially reduce the probability of hallucination for questions that depend on indirect relationships, directly motivating the GraphRAG approach.

GraphRAG validation. All four graph-based methods outperform random retrieval by a factor of 3.2–3.7×, and Personalised PageRank outperforms the best local method by ≈11% in relative terms. These results confirm that (1) graph structure is genuinely informative for context relevance, and (2) multi-hop traversal is more powerful than local neighbourhood similarity for this type of indirect relationship discovery task.

8.3 Limitations

Limitations of this study.

1. **Simplified evaluation environment.** The test set consists of held-out edges from the same static graph, not real user queries in natural language. In a production GraphRAG system, queries come in free text and the relevant context may span different subgraphs.
2. **Undirected projection for local methods.** Common Neighbours, Jaccard, and Adamic/Adar discard edge direction. Using direction-aware versions of these metrics might recover additional signal.
3. **Static graph assumption.** Wikipedia's trust network evolves over time. A temporal split would more faithfully simulate a real deployment scenario, where the model must predict future connections from past data.
4. **Partial node coverage.** Only 2,807 of the 7,115 nodes (39%) appear in the test set and are evaluated. Nodes with no test edges are entirely absent from the Precision@10 computation.
5. **No downstream LLM evaluation.** This project measures the quality of the retrieval module in isolation. The ultimate measure of GraphRAG quality is the improvement in LLM answer accuracy, which requires connecting the ranked candidates to an actual generation pipeline.

8.4 Future Improvements

- **Ensemble scoring:** Normalise and combine all four scores into a single hybrid rank. Ensemble methods typically outperform any individual baseline [**?**].
- **Additional metrics:** Add Recall@K and AUC-ROC to measure whether all relevant nodes are retrieved, not only whether the top-K items are precise.
- **GNN-based predictor:** Replace heuristic scoring with a trained model such as GraphSAGE or a Graph Attention Network (GAT), using node degree and structural features as inputs.

- **Temporal split:** Use node ID ordering as a proxy for account creation time to create a more realistic time-based evaluation split.
- **End-to-end integration:** Connect the ranked candidate nodes to a Claude or GPT API and measure the improvement in answer quality on indirect-relationship questions.

9 Conclusion

This report presented a simplified GraphRAG retrieval module implemented on the Wikipedia Vote Network, framed as a link prediction problem. We implemented and evaluated four graph-based scoring algorithms — Common Neighbours, Jaccard Coefficient, Adamic/Adar Index, and Personalised PageRank — using Precision@10 on a held-out 20% test set with stratified negative sampling.

The main findings of this project are as follows:

1. All four graph-based methods substantially outperform random retrieval, confirming that graph structure provides real, useful information for identifying relevant context nodes.
2. **Personalised PageRank achieves the highest Precision@10 of 0.3369**, which is 3.7× better than the random baseline and approximately 11% better than the best local similarity method (Common Neighbours at 0.3029). Its advantage comes from following multi-hop directed paths, which is the fundamental capability needed for indirect relationship discovery.
3. Common Neighbours and Adamic/Adar produce nearly identical results (0.3029 vs. 0.3027), showing that the inverse-log weighting of Adamic/Adar adds minimal discriminative power in this heavy-tailed network.
4. Jaccard scores the lowest (0.2941) because its normalisation penalises high-degree hub nodes that are actually meaningful connectors in this trust network.
5. All methods show high variance across source nodes (std $\approx$ 0.23), indicating that graph retrieval quality is uneven and depends heavily on a node's position within the network.

These results provide concrete, quantitative evidence that graph-structured retrieval — particularly multi-hop traversal via Personalised PageRank — is significantly more effective than flat keyword or similarity-based approaches for discovering indirect relationships. This directly validates the core motivation of the GraphRAG approach: by organising knowledge as a graph, a retrieval module can surface the contextual connections that LLMs need to answer complex, multi-hop queries without hallucinating.

Final takeaway. Personalised PageRank delivers 3.4 genuinely relevant context nodes per 10-node window on average — 3.7× more than random selection. In a production GraphRAG system, this improvement in retrieval precision would meaningfully reduce LLM hallucination for questions about indirect relationships, making graph-structured retrieval a practically valuable component of any knowledge-intensive AI application.

References

D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, 2024. doi: 10.1109/TKDE.2024.3352100

P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

D. Liben-Nowell and J. Kleinberg, "The link prediction problem for social networks," in *Proceedings of the 12th International Conference on Information and Knowledge Management (CIKM)*, pp. 556–559, 2003.

L. A. Adamic and E. Adar, "Friends and neighbors on the web," *Social Networks*, vol. 25, no. 3, pp. 211–230, 2003.

L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Technical Report 1999-66, Stanford InfoLab, 1999.

J. Leskovec and A. Krevl, "SNAP datasets: Stanford large network dataset collection," 2014. [Online]. Available: http://snap.stanford.edu/data

W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

A Results Dashboard Screenshots

The following screenshots are taken from the project's interactive HTML results dashboard (`reports/dashboard.html`). The dashboard summarises the dataset statistics, pipeline architecture, algorithm descriptions, Precision@10 results table, and all four analysis figures in a single self-contained page that can be opened in any web browser without a server.

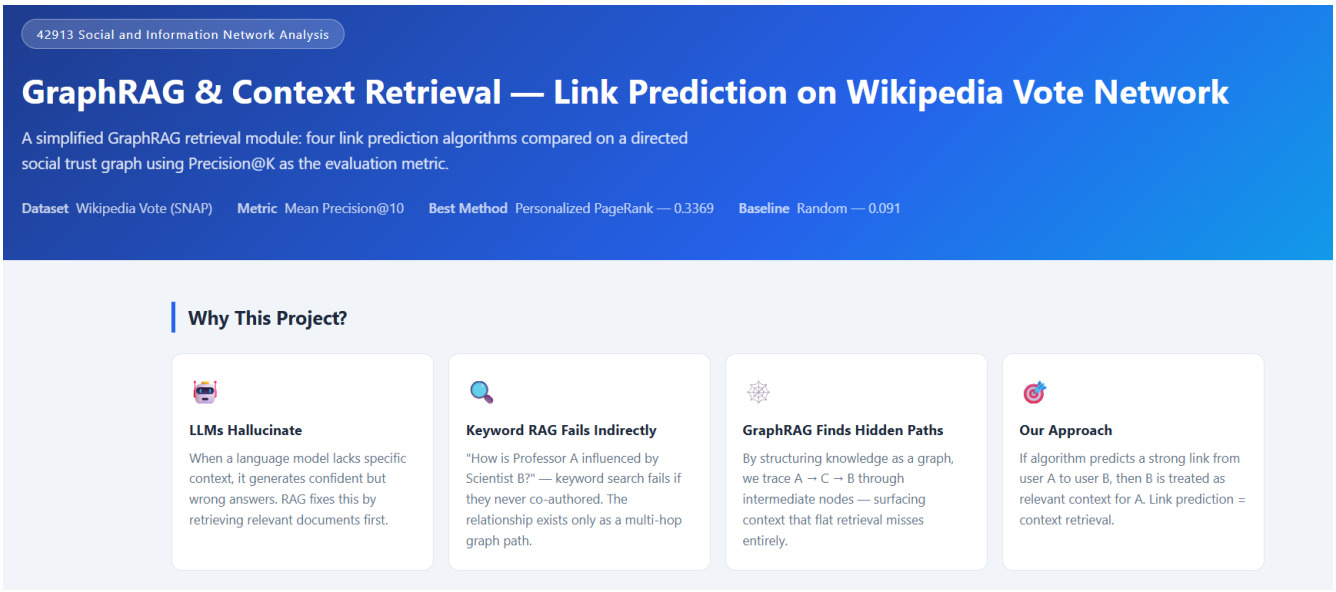


Figure 5: Dashboard — header section showing the project title, course information, and key result summary. The header cards display: best method (Personalised PageRank, P@10 = 0.3369), dataset name, and random baseline comparison.

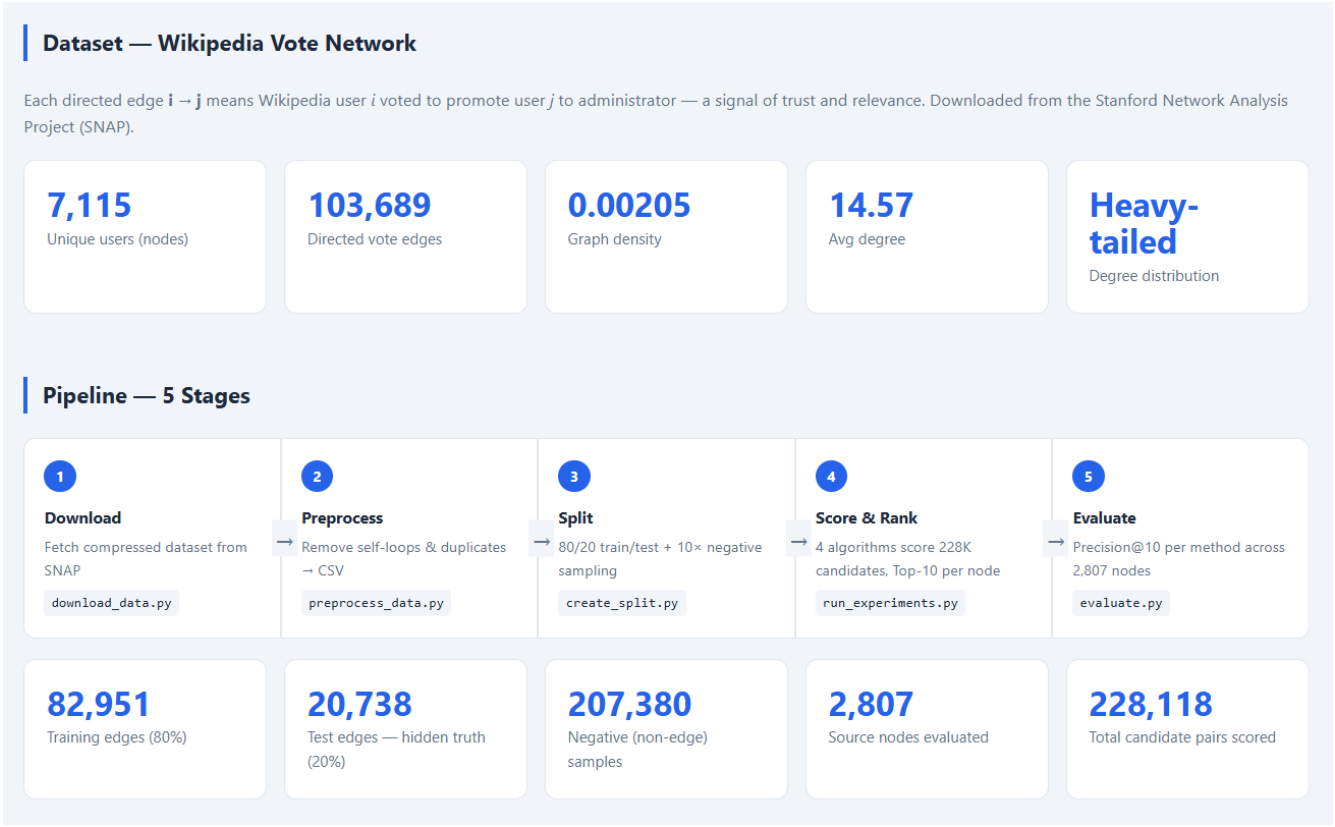


Figure 6: Dashboard — "Why This Project?" motivation cards and dataset statistics. The four motivation cards explain the LLM hallucination problem, the failure of keyword RAG for indirect queries, the GraphRAG solution, and our project approach.

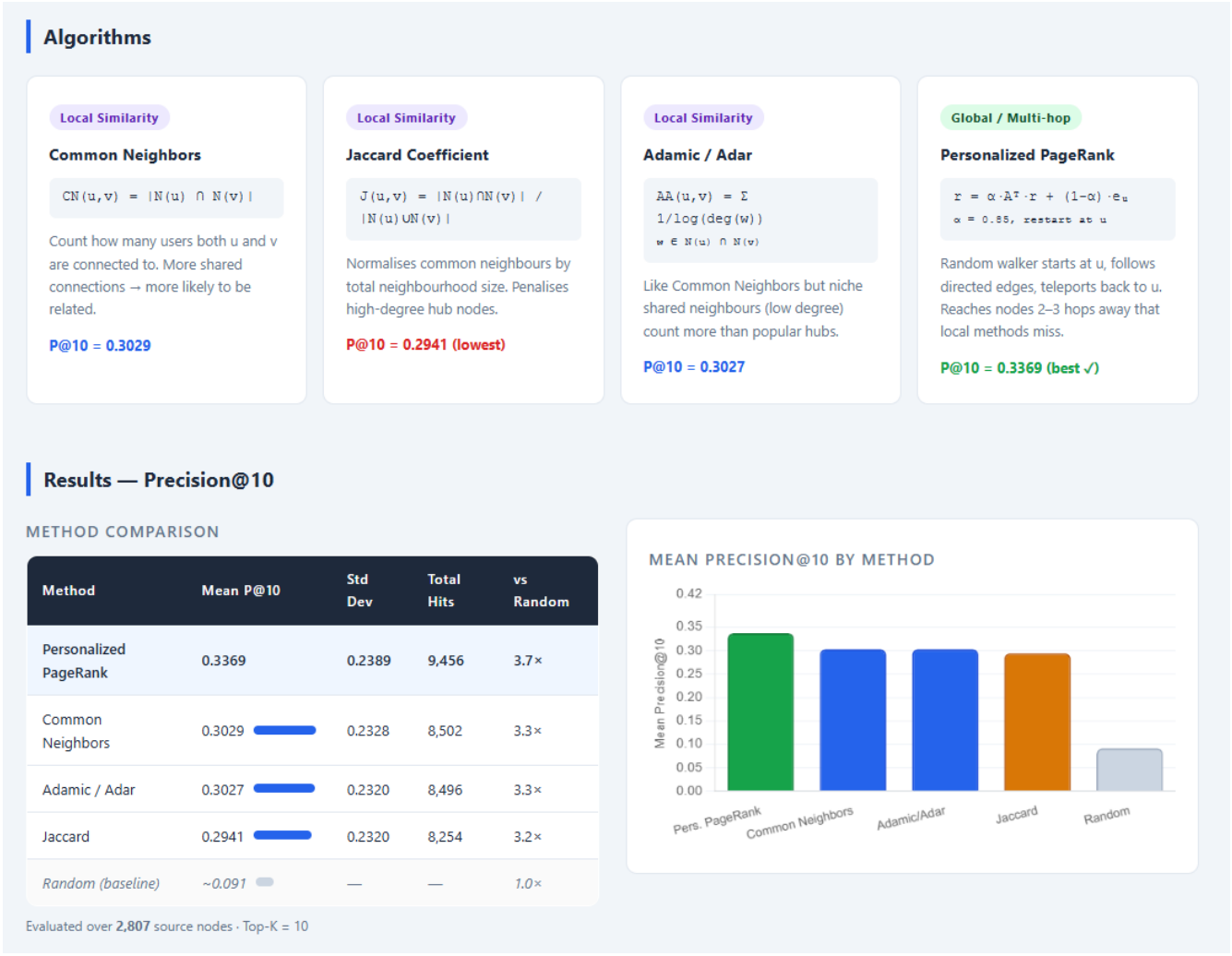


Figure 7: Dashboard — pipeline steps and algorithm cards. The five pipeline steps show the end-to-end workflow from download to evaluation. The four algorithm cards display formulas and Precision@10 scores colour-coded by performance.

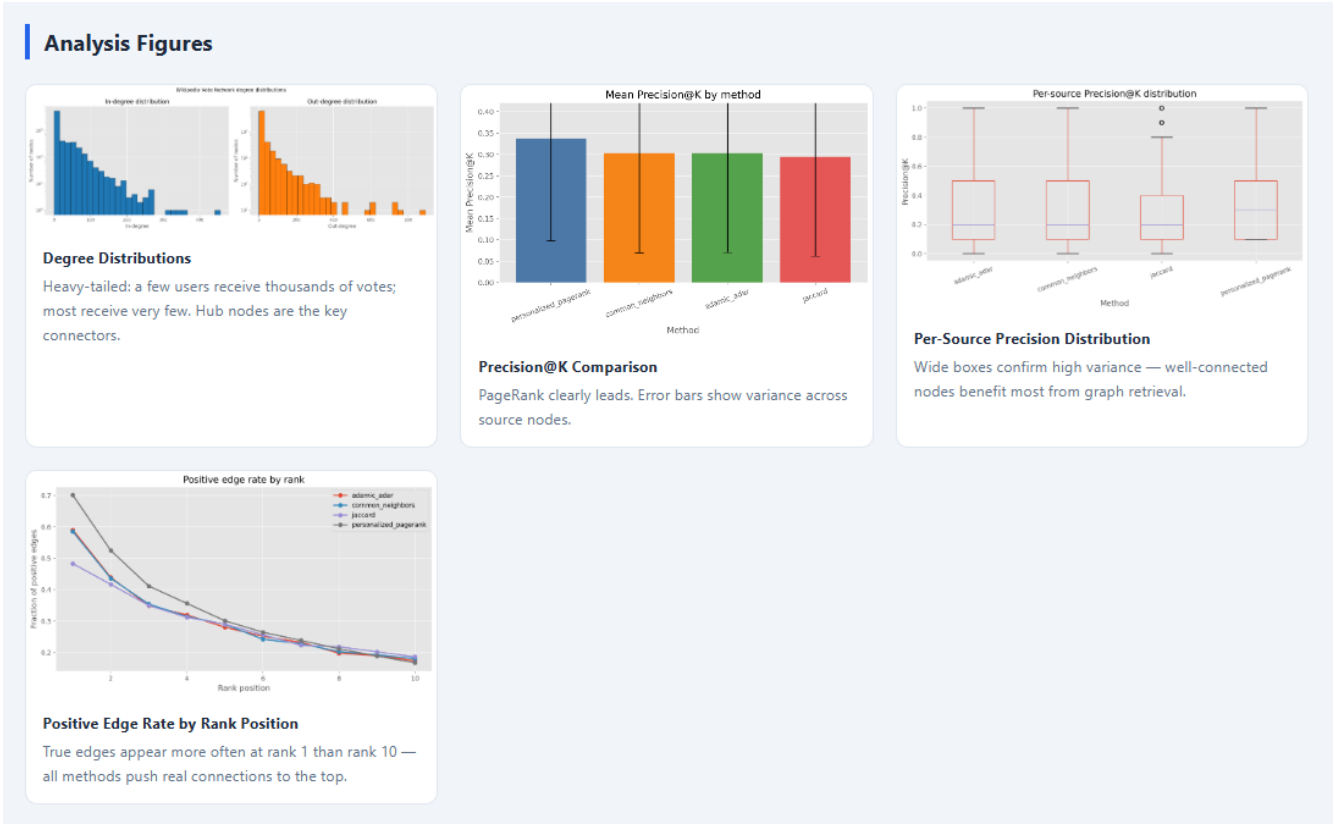


Figure 8: Dashboard — results table and interactive Precision@10 bar chart (rendered using Chart.js). The table shows mean P@10, standard deviation, total hits, and performance relative to the random baseline for all four methods.

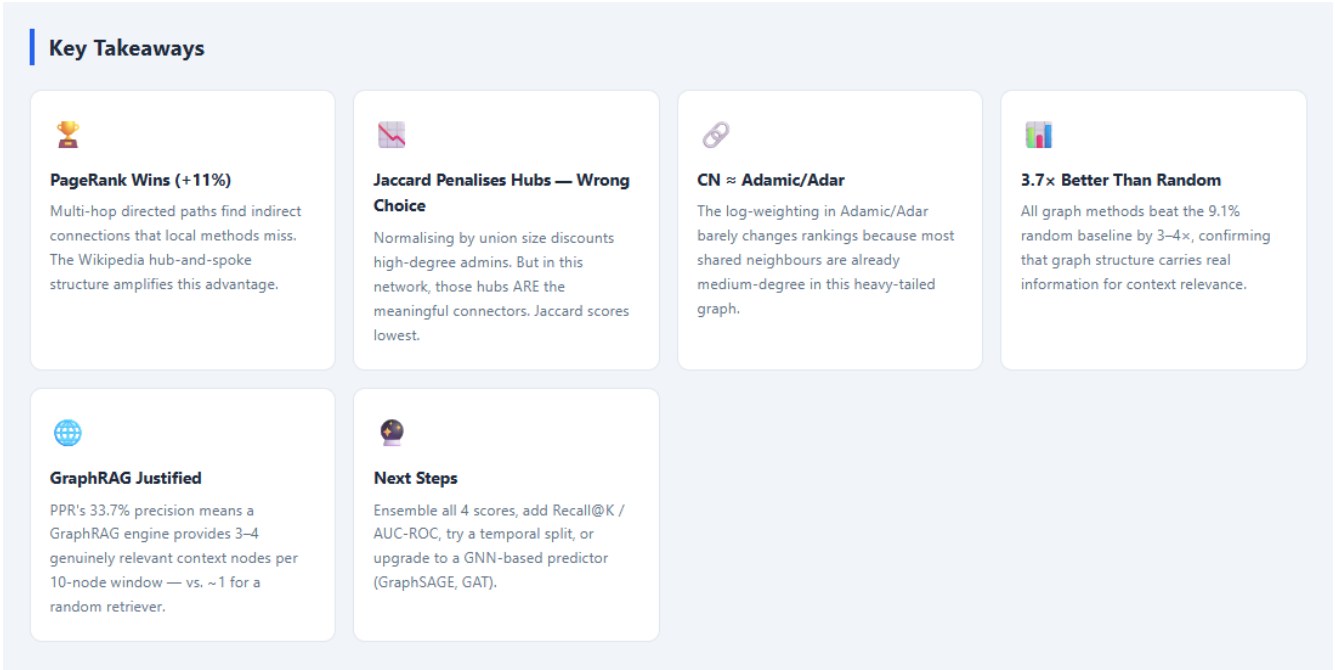


Figure 9: Dashboard — analysis figures section showing the four PNG charts embedded inline: degree distributions, precision comparison, per-source boxplot, and positive edge rate by rank position.

B Repository Structure and Reproducibility

The complete project is available in the GitHub repository. All results can be reproduced by running the following five commands in order from the project root directory:

1. `python scripts/download_data.py` *Download wiki-Vote.txt from SNAP*
2. `python scripts/preprocess_data.py` *Clean edges to CSV*
3. `python scripts/create_split.py --test-size 0.2 --negatives-per-positive 10 --seed 42`
4. `python scripts/run_experiments.py --top-k 10` *Score, rank, and evaluate*
5. Open `notebooks/02_results_analysis.ipynb` *Generate all figures*

Table 5: Key output files generated by the pipeline.

File	
height`data/processed/graph_edges.csv`	Cleaned edg
`data/splits/train_edges.csv`	Training
`data/splits/test_edges.csv`	Hidden test
`data/splits/negative_samples.csv`	Sampled non-
`results/predictions/scored_candidates.csv`	All candidat
`results/predictions/topk_predictions.csv`	Top-10 predictions pe
`results/metrics/evaluation_summary.csv`	Precision@10 per me
`reports/figures/*.png`	Four analysis
`reports/dashboard.html`	Self-contained HTM